

SERVER SCRIPT

GET_SHIFT_DETAILS

Script Type: API
Event Frequency: All
DocType Event: Before Insert
API Method: get_shift_details

Script

```
# Script: get_shift_details
# Logic: Fetches production, consumption, and wastage details for a specific shift

posting_date = frappe.form_dict.get('posting_date')
shift = frappe.form_dict.get('shift')
unit = frappe.form_dict.get('unit')

if not posting_date or not shift:
    frappe.throw("Posting Date and Shift are required")

# 1. Fetch matching submitted Roll Production Entries
rpe_filters = {
    "docstatus": 1,
    "posting_date": posting_date,
    "shift": shift,
}
if unit: rpe_filters["unit"] = unit

rpe_docs = frappe.get_all("Roll Production Entry",
    filters=rpe_filters,
    fields=["name", "work_order", "shift_batch_number"]
)

# EARLY ESCAPE: If no production entries exist for this shift/unit/date, return empty result
if not rpe_docs:
    frappe.response['message'] = {
        "production_items": [],
        "consumption_items": [],
        "wastage_items": [],
        "production_attributes": [],
        "base_batch_no": ""
    }
else:
    valid_work_orders = []
    base_batch_no = ""
    wastage_list = []
    production_attributes = []
    seen_attrs = set()
```

```

for rpe in rpe_docs:
    if not base_batch_no and rpe.get("shift_batch_number"):
        base_batch_no = rpe.get("shift_batch_number")

    rpe_doc = frappe.get_doc("Roll Production Entry", rpe.name)

    # AGGRESSIVE ATTRIBUTE EXTRACTION
    for f in rpe_doc.meta.get_table_fields():
        rows = rpe_doc.get(f.fieldname) or []
        for row in rows:
            q = row.get("quality")
            c = row.get("colour") or row.get("color")
            g = row.get("gsm")

            if q and c and g:
                attr_key = (str(q).strip().upper(), str(c).strip().upper(), str(g).strip().upper())
                if attr_key not in seen_attrs:
                    production_attributes.append({
                        "quality": q, "colour": c, "gsm": g
                    })
                    seen_attrs.add(attr_key)

            wo = row.get("work_order") or rpe_doc.get("work_order")
            if wo and wo not in valid_work_orders:
                valid_work_orders.append(wo)

# 2. Fetch Stock Entries for items produced
se_filters = {
    "docstatus": 1,
    "purpose": "Manufacture",
}
if valid_work_orders:
    se_filters["work_order"] = ["in", valid_work_orders]
else:
    se_filters["posting_date"] = posting_date

stock_entries = frappe.get_all("Stock Entry",
    fields=["name", "work_order", "from_warehouse", "to_warehouse", "bom_no"],
    filters=se_filters
)

production_map = {}
consumption_map = {}

for se in stock_entries:
    for item in frappe.get_all("Stock Entry Detail",

```

```

fg_items = frappe.get_all('Stock Entry Detail',
    filters={"parent": se.name, "is_finished_item": 1},
    fields=["item_code", "item_name", "qty", "uom"]
)

for fg in fg_items:
    key = (fg.item_code, se.work_order, se.to_warehouse)
    if key not in production_map:
        production_map[key] = {
            "work_order": se.work_order,
            "stock_entry": se.name,
            "item_code": fg.item_code,
            "item_name": fg.item_name,
            "produced_qty": 0.0,
            "fg_warehouse": se.to_warehouse
        }

    # Explicit assignment for RestrictedPython compliance
    p_row = production_map[key]
    p_row["produced_qty"] = p_row["produced_qty"] + float(fg.qty or 0)

    # ATTR SPLITTING FROM ITEM NAME
    if not production_attributes or len(production_attributes) < 5:
        nm = str(fg.item_name).upper()
        if " GSM" in nm:
            g_part = nm.split(" GSM")[0].split(" ")[-1]
            if g_part.isdigit():
                m_parts = nm.replace("NON WOVEN FABRIC ", "").split(" " + g_part + " GSM")[0].strip().split(" ")
                if len(m_parts) >= 2:
                    q_val = m_parts[0]
                    c_val = " ".join(m_parts[1:])
                    k = (q_val, c_val, g_part)
                    if k not in seen_attrs:
                        production_attributes.append({
                            "quality": q_val, "colour": c_val, "gsm": g_part
                        })
                        seen_attrs.add(k)

    # BOM Consumption
    bom_no = se.bom_no or frappe.db.get_value("Work Order", se.work_order, "bom_no")
    if bom_no:
        bom_qty = float(frappe.db.get_value("BOM", bom_no, "quantity") or 1) or 1
        bom_items = frappe.get_all("BOM Item",
            filters={"parent": bom_no},
            fields=["item_code", "item_name", "qty", "stock_uom"]
        )
        for rm in bom_items:

```

SERVER SCRIPT

GET_SHIFT_DETAILS

```
req_qty = (float(rm.qty) / bom_qty) * float(fg.qty)
if rm.item_code not in consumption_map:
    consumption_map[rm.item_code] = {
        "item_code": rm.item_code, "item_name": rm.item_name, "uom": rm.stock_uom,
        "standard_consumption": 0.0, "actual_consumption": 0.0
    }
# Explicit assignment for RestrictedPython compliance
c_row = consumption_map[rm.item_code]
c_row["standard_consumption"] = c_row["standard_consumption"] + req_qty
c_row["actual_consumption"] = c_row["actual_consumption"] + req_qty

frappe.response['message'] = {
    "production_items": list(production_map.values()),
    "consumption_items": list(consumption_map.values()),
    "wastage_items": [],
    "production_attributes": production_attributes,
    "base_batch_no": base_batch_no
}
```

Request Limit:

5

Time Window (Seconds):

86400